

	Propuesta de Proyecto	Código	
		Página	1 de 20

**SISTEMA DE CONTROL DE ASISTENCIA CON
RECONOCIMIENTO FACIAL**

INTEGRANTES

ANDRES FELIPE JAIMES MUÑOZ

GILBERTH ASDRUBAL GARCIA LIZCANO

UNIVERSIDAD DE PAMPLONA

PAMPLONA, 1 de abril de 2026

	Propuesta del Proyecto	Código	
		Página	2 de 20

1. INFORMACIÓN GENERAL DEL PROYECTO

Línea de Investigación: Inteligencia Artificial	
Lugar de ejecución del proyecto: Universidad de Pamplona	
Duración del proyecto (en meses): 3 meses (12 semanas)	
Área: Ingeniería de Sistemas	
Línea estratégica: Innovación Tecnológica	
Descriptor/es/Palabras claves: reconocimiento facial, asistencia automática, OpenCV, inteligencia artificial	

2. RESUMEN DEL PROYECTO

Este proyecto propone el desarrollo de un sistema automatizado de control de asistencia basado en reconocimiento facial, orientado a entornos académicos. El sistema emplea visión por computador para capturar imágenes en tiempo real, extraer características biométricas (encodings faciales) y compararlas con una base de datos previamente registrada.

A diferencia de los métodos tradicionales, esta solución reduce significativamente el tiempo de registro y elimina riesgos de suplantación de identidad. El sistema será implementado en Python utilizando librerías especializadas como OpenCV y face_recognition, integrando además una interfaz gráfica desarrollada en Tkinter y una base de datos relacional.

Como resultado, se espera obtener un prototipo funcional capaz de registrar asistencia en menos de 2 segundos por estudiante, con una precisión superior al 95%, cumpliendo además con las normativas de protección de datos personales (Ley 1581 de 2012 en Colombia).

3. PLANTEAMIENTO DEL PROBLEMA- HIPÓTESIS

Actualmente, el control de asistencia en muchas instituciones se hace de forma manual, lo cual puede generar problemas como pérdida de tiempo en clase, errores al anotar y hasta suplantación de identidad.

Hipótesis:

Si se implementa un sistema de reconocimiento facial para el control de asistencia, entonces se

	Propuesta del Proyecto	Código	
		Página	3 de 20

reducirá el tiempo de marcación en al menos un 80% en comparación con el método manual, y se eliminarán los errores por suplantación de identidad, siempre que las condiciones de iluminación y captura sean adecuadas.

4. OBJETIVO GENERAL

Desarrollar un sistema que permita registrar la asistencia de estudiantes utilizando reconocimiento fácil, de manera automática y en tiempo real.

4.1. Objetivos específico

- Crear un módulo de registro donde los estudiantes puedan inscribirse con sus datos personales y al menos 5 imágenes de su rostro (diferentes ángulos e iluminación).
- Implementar el reconocimiento facial utilizando OpenCV y la librería `face_recognition` en Python.
- Diseñar una base de datos donde se guarde la información de los estudiantes y la asistencia.
- Integrar todo el sistema para que funcione correctamente en tiempo real.
- Realizar pruebas de precisión, rendimiento y usabilidad para verificar que el sistema cumpla con los criterios de éxito definidos.

4.2. Requerimientos del sistema

Con base en el problema y los objetivos, se establecen los siguientes requerimientos funcionales y no funcionales para el prototipo.

4.2.1 Requerimientos funcionales (RF)

Código	Funcionalidad	Descripción
RF-01	Registrar estudiante	Registrar un estudiante con: nombre completo, identificación (código) y curso/asignatura (obligatorio) e imagen(obligatorio).
RF-02	Generar encodings faciales	Extraer el vector de características faciales (encoding) de cada imagen usando la librería

	Propuesta del Proyecto	Código	
		Página	4 de 20

		face_recognition.
RF-03	Asociar encodings al estudiante	Guardar los encodings en la base de datos vinculados al ID del estudiante.
RF-04	Actualizar datos del estudiante	Permitir modificar nombre, curso o volver a capturar imágenes (reemplazando los encodings anteriores).
RF-05	Eliminar estudiante	Borrar del sistema todos los datos del estudiante: información personal, encodings e historial de asistencia.
RF-06	Activar cámara en tiempo real	Abrir el flujo de video desde la cámara configurada y mostrarlo en la interfaz.
RF-07	Detectar rostros en video	Detectar automáticamente todos los rostros presentes en cada fotograma usando OpenCV.
RF-08	Comparar con la base de datos	Comparar el encoding obtenido con todos los encodings almacenados en la base de datos.
RF-09	Identificar coincidencia con el estudiante	Recuperar los datos del estudiante correspondiente al encoding que produjo la coincidencia.
RF-10	Registrar asistencia automática	Marcar automáticamente la asistencia del estudiante identificado.
RF-11	Guardar datos de asistencia	Almacenar en la base de datos: ID del estudiante, fecha, hora exacta y

	Propuesta del Proyecto	Código	
		Página	5 de 20

		asignatura.
RF-12	Evitar asistencias duplicadas	Un mismo estudiante solo puede tener una asistencia por día para la misma asignatura.
RF-13	Registro manual de asistencia	Permitir al docente marcar asistencia de forma manual en caso de fallo del reconocimiento.
RF-14	Consultar asistencias	Filtrar y visualizar asistencias por: estudiante, fecha, curso/asignatura.
RF-15	Generar reportes	Mostrar los resultados en formato tabla y permitir la exportación a archivo CSV.
RF-16	Mostrar historial de asistencias	Visualizar el registro completo de asistencias de cada estudiante.
RF-17	Interfaz gráfica principal	Proveer una interfaz con Tkinter que permita: (a) registrar estudiantes, (b) iniciar reconocimiento facial, (c) consultar asistencias.
RF-18	Mostrar resultado en pantalla	Durante el reconocimiento, indicar el nombre del estudiante reconocido y su estado (Registrado / no registrado).
RF-19	Notificaciones visuales	Mostrar mensajes como "Asistencia registrada exitosamente" o "Estudiante no reconocido".
RF-20	Inicializar base de datos	Crear automáticamente las tablas necesarias la primera

	Propuesta del Proyecto	Código	
		Página	6 de 20

		vez que se ejecuta la aplicación.
RF-21	Manejo de errores	Capturar y mostrar mensajes claros ante fallos como cámara no disponible, base de datos dañada, etc.

4.2.2 Requerimientos no funcionales (RNF)

Código	Categoría	Requerimiento
RNF-01	Plataforma	Aplicación de escritorio para PC.
RNF-02	Tecnología	Desarrollada en Python 3.
RNF-03	Interfaz	Interfaz gráfica construida con Tkinter (biblioteca estándar).
RNF-04	Compatibilidad	Funcionamiento probado en Windows; compatible también con Linux sin modificaciones.
RNF-05	Rendimiento	Tiempo entre detección y marcación de asistencia ≤ 2 segundos.
RNF-06	Fluidez	Fluidez Procesamiento de video en tiempo real a un mínimo de ~15 FPS.
RNF-07	Capacidad	Soporte para al menos 50 estudiantes registrados sin degradación.
RNF-08	Precisión	Tasa de aciertos ≥ 95 % en condiciones normales de iluminación.
RNF-09	Tasa de error	Falsos positivos (marcar a

	Propuesta del Proyecto	Código	
		Página	7 de 20

		quien no es) menor al 2 %.
RNF-10	Privacidad	No se almacenan imágenes originales, únicamente los encodings vectoriales.
RNF-11	Cumplimiento legal	Apego a la Ley 1581 de 2012 (Colombia) para el tratamiento de datos personales.
RNF-12	Control de acceso	Acceso restringido mediante un PIN simple para uso exclusivo del docente.
RNF-13	Seguridad BD	Archivo de base de datos SQLite protegido por los permisos del sistema operativo.
RNF-14	Usabilidad	Interfaz intuitiva para un docente sin experiencia técnica.
RNF-15	Eficiencia de uso	El flujo principal (inicio de cámara y marcación) no requiere más de 3 clics.
RNF-16	Retroalimentación	Mensajes de error y éxito claros y visibles.
RNF-17	Modularidad	Código fuente organizado en módulos: interfaz, lógica de reconocimiento, acceso a datos.
RNF-18	Documentación	Código comentado con docstrings y un README con instrucciones de instalación y uso.
RNF-19	Limitación GUI	Interfaz limitada a los widgets estándar de Tkinter (sin

	Propuesta del Proyecto	Código	
		Página	8 de 20

		diseño web moderno).
RNF-20	Escalabilidad visual	Diseñada para un aula grande (máximo 50 estudiantes).

5. METODOLOGÍA

El proyecto se desarrollará siguiendo un enfoque **iterativo e incremental** (adaptación de Scrum a 3 sprints, uno por mes). Cada sprint incluye análisis, desarrollo, pruebas internas y revisión.

5.1. Fase de análisis (Semana 1 a 3)

- Estudio del problema y definición de requisitos funcionales y no funcionales.
- Investigación de herramientas: Python, OpenCV, face_recognition, bases de datos.
- Diseño de la base de datos y arquitectura de componentes.
- **Entregable:** Documento de requisitos, diagramas E-R y de componentes.

5.2. Fase de diseño (semana 4 a 8)

- Implementación del módulo de registro (captura facial y almacenamiento de encodings).
- Implementación del módulo de reconocimiento en tiempo real.
- Desarrollo del registro automático de asistencia con control de duplicados (un solo registro por día y por asignatura).
- Construcción de una interfaz de usuario simple (por consola o gráfica con Tkinter).
- Integración de todos los módulos.
- **Entregable:** Prototipo funcional en entorno de pruebas.

5.3. Fase de pruebas y documentación (Semanas 9 a 12)

- Pruebas de precisión (variación de iluminación, ángulos, obstrucciones parciales).
- Pruebas de rendimiento (tiempo de detección y comparación por rostro).
- Corrección de errores y optimización de parámetros (umbral de similitud).

	Propuesta del Proyecto	Código	
		Página	9 de 20

- Elaboración del informe final, manual de usuario y manual técnico.
- Preparación de la presentación y video demostrativo.
- **Entregable:** Informe final, código fuente documentado, manuales, presentación.

6. COMPONENTES CLAVES

Componente	Descripción
Cámara	Permite capturar el rostro de los estudiantes en tiempo real mediante el uso de OpenCV.
Módulo de reconocimiento	Se encarga de detectar rostros en tiempo real y compararlos con los registros previamente capturados, utilizando la librería de reconocimiento facial.
Almacenamiento	En la fase actual, el sistema utiliza una estructura de carpetas para almacenar imágenes de los estudiantes y archivos CSV para registrar la asistencia. En futuras versiones se migrará a una base de datos para mejorar la gestión de la información.
Aplicación / Interfaz	Permite al usuario (docente) interactuar con el sistema: registrar estudiantes, iniciar el reconocimiento facial y visualizar el proceso de asistencia mediante una interfaz gráfica construida con Tkinter.
Algoritmo	Procesa y compara los rostros utilizando la librería face_recognition, la cual se basa en dlib y modelos de redes neuronales para generar y comparar encodings faciales.

6.1. DESCRIPCIÓN DE LA INTERACCIÓN ENTRE LOS COMPONENTES

	Propuesta del Proyecto	Código	
		Página	10 de 20

1. **Registro:** El estudiante ingresa su nombre y el sistema captura múltiples imágenes de su rostro desde diferentes ángulos utilizando la cámara. Estas imágenes se almacenan en una estructura de carpetas organizada por estudiante.
2. **Uso diario:** El docente inicia la aplicación. La cámara captura el rostro de los estudiantes en tiempo real. El sistema genera encodings faciales a partir de las imágenes capturadas y los compara con los encodings generados a partir de las imágenes almacenadas. Si se encuentra una coincidencia, se identifica al estudiante y se registra su asistencia en un archivo CSV con fecha y hora.

La comparación se realiza utilizando una métrica de distancia (por ejemplo, distancia euclidiana), considerando una coincidencia válida cuando el valor se encuentra por debajo de un umbral definido (ej. 0.6).

3. **Consulta:** El docente puede acceder a los registros de asistencia almacenados en el archivo CSV, los cuales pueden ser filtrados o exportados en fases posteriores del desarrollo.

Nota: En la fase actual del desarrollo, el sistema almacena temporalmente imágenes faciales para facilitar el entrenamiento y validación del modelo. En versiones posteriores, estas imágenes serán reemplazadas por encodings faciales almacenados en base de datos, en cumplimiento con los requisitos de privacidad y diseño del sistema.

6.2 Mockups.

Los mockups del sistema fueron diseñados utilizando la herramienta Figma, con el objetivo de representar visualmente la interfaz gráfica de la aplicación antes de su implementación final.

Un mockup es una representación visual de alta fidelidad que muestra cómo será la apariencia del sistema, incluyendo elementos como botones, formularios, colores y distribución de componentes, sin incluir aún la funcionalidad real.

A través de estos diseños se busca validar la experiencia del usuario, la organización de la información y la interacción con el sistema, permitiendo realizar mejoras antes del desarrollo completo.

Los mockups desarrollados pueden visualizarse en el siguiente enlace:

<https://www.figma.com/make/IGqmdswX3HxoVpCnVqD9B9/Interfaz-de-Control-de-Asistencia?fullscreen=1&t=xFDmrAgQ5xKJ4ZON-1>

	Propuesta del Proyecto	Código	
		Página	11 de 20

- **La pantalla principal** permite al usuario acceder a las funcionalidades principales del sistema, como el registro de estudiantes y el inicio del reconocimiento facial. Su diseño es simple e intuitivo, facilitando la navegación del docente.
- **En esta pantalla de registro** el usuario puede ingresar los datos del estudiante y activar la cámara para capturar múltiples imágenes del rostro. El diseño está enfocado en guiar al usuario durante el proceso de registro.
- **En esta pantalla de reconocimiento**, esta interfaz muestra el flujo de video en tiempo real y permite identificar a los estudiantes mediante reconocimiento facial. También muestra información visual como el nombre detectado.
- **Pantalla de reportes**, permite visualizar los registros de asistencia almacenados, facilitando la consulta por parte del docente.

6.4 Diagramas

DIAGRAMA DE FLUJO DE DATOS

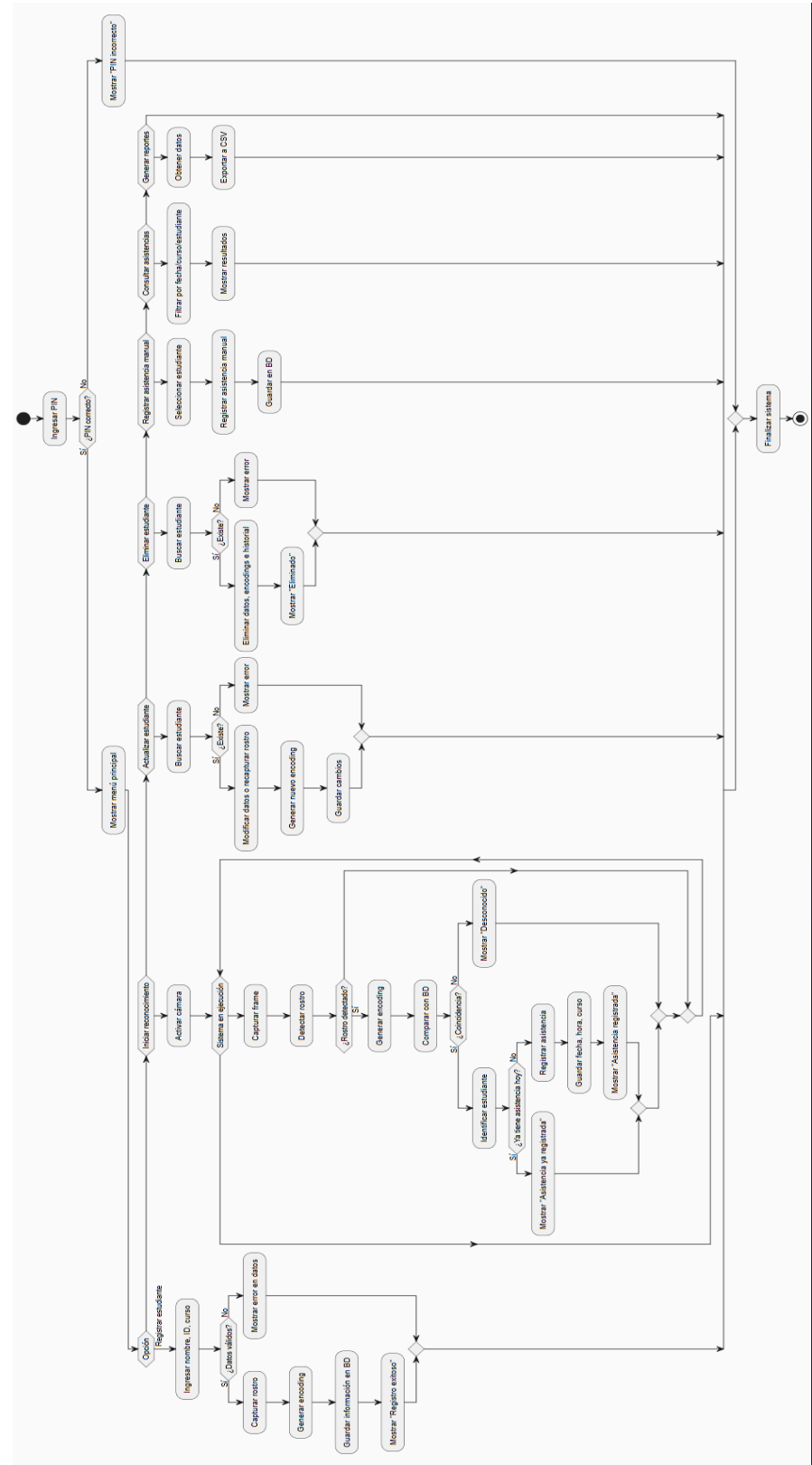
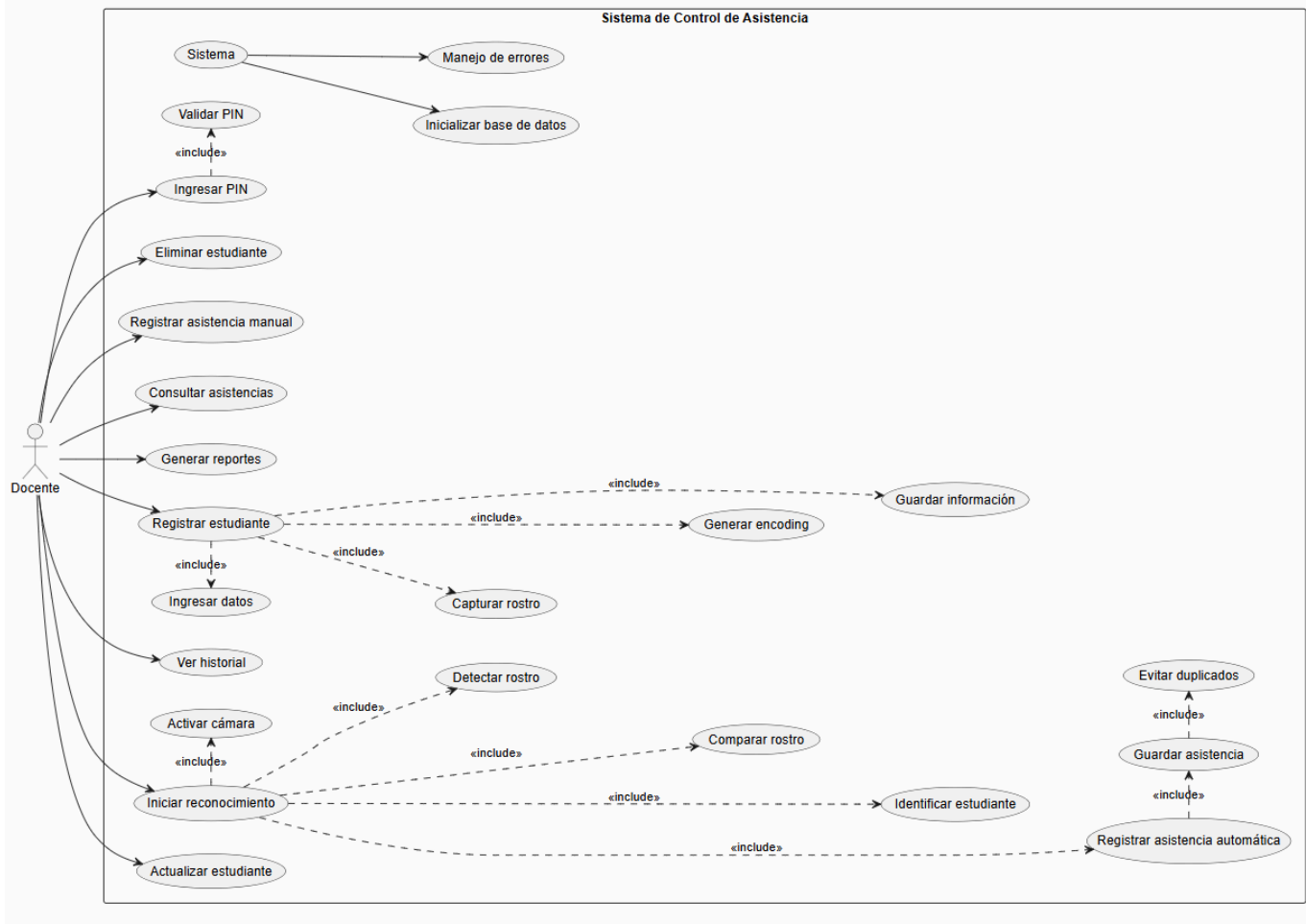


DIAGRAMA DE CASOS DE USO



	Propuesta del Proyecto	Código	
		Página	14 de 20

6.5 Herramientas.

Categoría	Herramienta	Descripción
Lenguaje	Python 3.*	Lenguaje principal del sistema
Visión artificial	OpenCV	Captura de video y detección de rostros
Reconocimiento facial	face_recognition	Generación y comparación de encodings faciales
Motor IA	dlib	Procesamiento de características faciales
Interfaz gráfica	Tkinter	Construcción de la aplicación de escritorio
Base de datos (prototipo)	CSV	Almacenamiento temporal de asistencias
Manejo de imágenes	Pillow	Procesamiento básico de imágenes
Cálculo numérico	NumPy	Manejo de vectores y operaciones matemáticas
Control de versiones	Git	Gestión del código fuente
Entorno de desarrollo	venv	Aislamiento de dependencias
Modelos de reconocimiento	face_recognition_models	Modelos preentrenados para detección y encoding facial

	Propuesta del Proyecto	Código	
		Página	15 de 20

Nota: En fases posteriores se migrará el almacenamiento de CSV a SQLite para cumplir con los requerimientos de persistencia del sistema.

6.6 Estructura del proyecto

	Propuesta del Proyecto	Código	
		Página	16 de 20

```

asistencia/
|
|— main.py
|
|— presentation/                # Interfaz gráfica (Tkinter)
|   |— main_window.py
|   |— register_view.py
|   |— attendance_view.py
|   └— reports_view.py
|
|— application/                # Lógica de negocio
|   |— student_service.py
|   |— attendance_service.py
|   |— report_service.py
|   |— camera_service.py
|   └— auth_service.py
|
|— domain/                    # Modelos del sistema
|   |— student.py
|   └— attendance.py

```

```

|
|— infrastructure/            # Implementación técnica
|   |— database/
|       |— db_manager.py
|       |— student_repo.py
|       |— attendance_repo.py
|       └— schema.sql
|   └— recognition/
|       |— face_engine.py
|       └— encoder.py
|
|— shared/                    # Utilidades
|   |— validators.py
|   └— csv_exporter.py
|
|— config/
|   └— settings.py
|
|— data/
|   |— fotos/
|   └— asistencia.csv
|
|— tests/
|
|— requirements.txt
└— README.md

```



	Propuesta del Proyecto	Código	
		Página	17 de 20

El proyecto se organiza siguiendo una arquitectura modular por capas, separando la interfaz de usuario, la lógica de negocio, el reconocimiento facial y la persistencia de datos. Esta estructura facilita la escalabilidad, el mantenimiento y la integración de nuevos componentes.

La arquitectura implementada se basa en principios de separación de responsabilidades, permitiendo desacoplar los módulos de reconocimiento facial, interfaz gráfica y almacenamiento de datos.

Nota: Actualmente, el sistema utiliza almacenamiento en archivos CSV como solución temporal. En futuras fases se migrará a una base de datos (SQLite), lo que puede implicar ajustes en la estructura del proyecto para mejorar la organización y escalabilidad del sistema.

7. CRONOGRAMA DE ACTIVIDADES

Mes	Actividad Principal	Producto esperado
1 (Semanas 1-4)	Análisis, diseño y módulo de registro	Documento de requisitos, BD diseñada, módulo de registro funcional
2 (Semanas 5-8)	Reconocimiento, asistencia e interfaz	Prototipo que reconoce rostros y registra asistencia
3 (Semanas 9-12)	Pruebas, optimización y documentación	Informe final

7.1 Cronograma y descripción de actividades:

	Propuesta del Proyecto	Código	
		Página	18 de 20

Tabla 7.1. Cronograma y descripción de actividades

12 semanas												
ACTIVIDAD	1	2	3	4	5	6	7	8	9	10	11	12
I												
II												
III												

Nota: Agregue las filas necesarias

Descripción de actividades:

I. Análisis y diseño (mes 1)

- Investigación del estado del arte (reconocimiento facial, OpenCV, bases de datos).
- Definición de requisitos funcionales (registro, captura, comparación, marcación de asistencia) y no funcionales (tiempo de respuesta, precisión).
- Diseño de la base de datos (tablas: estudiantes, asistencias, rostros codificados).
- Selección de herramientas: Python, OpenCV, face_recognition, SQLite/MySQL.
- Diagrama de componentes y flujo de interacción.

II. Desarrollo del sistema

- Implementación del módulo de registro: captura de fotos, extracción de características faciales, almacenamiento en BD.
- Implementación del módulo de reconocimiento en tiempo real (cámara, detección, comparación).
- Desarrollo del registro automático de asistencia (fecha, hora, ID del estudiante).
- Creación de una interfaz simple (por consola o gráfica con Tkinter/PyQt).

	Propuesta del Proyecto	Código	
		Página	19 de 20

- Integración de todos los componentes.

III. Pruebas y documentación

- Pruebas unitarias y de integración.
- Pruebas de precisión con diferentes condiciones de iluminación, ángulos y distancias.
- Pruebas de rendimiento (tiempo de detección, falsos positivos/negativos).
- Corrección de errores y optimización.
- Elaboración del informe final (resultados, lecciones aprendidas, trabajo futuro).
- Redacción del manual de usuario y del manual técnico.
- Preparación de la presentación final.

8. CONSIDERACIONES ETICAS DE PRIVACIDAD

El sistema almacenará únicamente las características faciales (encodings) y no las imágenes originales, salvo consentimiento explícito del estudiante. Se cumplirá con la normativa de protección de datos personales aplicable en Colombia (Ley 1581 de 2012 y sus decretos reglamentarios). Los datos serán usados exclusivamente para el control de asistencia académica.

9. CRITERIOS DE ÉXITO (MÉTRICAS)

Métrica	Valor esperado
Precisión del reconocimiento (tasa de acierto)	≥ 95% en condiciones normales de iluminación
Tiempo de registro por estudiante	< 2 segundos
Capacidad de la base de datos	Mínimo 50 estudiantes
Reducción de tiempo de marcación	≥ 80% frente al método manual

	Propuesta del Proyecto	Código	
		Página	20 de 20

Tasa de falsos positivos (marcar a quien no es)	< 2%
---	------

10. RESULTADOS ESPERADOS

- Código fuente del sistema alojado en un repositorio público (GitHub).
- Prototipo funcional que pueda ser probado en un aula con al menos 10 estudiantes.
- Manual de instalación y uso (paso a paso).
- Informe técnico con resultados de pruebas y lecciones aprendidas.
- Video demostrativo (máximo 5 minutos) mostrando el flujo completo.